

GraphOne / FrontierAtlas - Technical Architecture & Scale Design

Production Design for 500,000+ Multi-Dimensional Intelligence Graph Ingestion

Executive Summary: This document details the production engineering architecture for GraphOne's global AI Intelligence Graph. The system is designed to continuously ingest, normalize, resolve, and link hundreds of thousands (500k+) of entity records across startups, products, research papers, real-time news signals, and job openings with high fault tolerance.

1. Scale Strategy: Acquiring 500,000+ Entities

To scale data acquisition to 500,000+ startups, products, and papers without manual intervention, the architecture transitions from a single async process to a Distributed Micro-Crawler Mesh:

- Distributed Event Backbone: Apache Kafka / RabbitMQ partition queues by domain tier (High-frequency news vs. Bulk directory scraping).
- Headless Crawler Swarm: Auto-scaling Kubernetes pods running Playwright Async and Scrapy clusters. IP rotation managed via residential proxy pools (BrightData/ScraperAPI) to bypass Cloudflare/DataDome captchas.
- Partitioned Ingestion: Partitioning by domain shards & seed directories ensures parallel workers crawl distinct namespace segments with zero lock contention.

2. Managing Context Windows (413) & Rate Limits (429)

Extracting structured JSON from massive web pages via LLMs requires strict context-window and rate-limit guardrails:

- 413 Payload Too Large Mitigation: Raw HTML DOMs are passed through an AST-based DOM Cleaner (stripping script/style tags, inline SVG, boilerplates) followed by a Semantic Token Slicer. Payload chunk sizes are clamped below model token limits (e.g. 12,000 chars) while preserving dense headers/content.
- 429 Rate Limit Mitigation: Implements a Leaky Bucket Rate Limiter combined with an Exponential Backoff + Jitter retry loop (delay = 2^{attempt} + $U(0, 1.5)\text{s}$).
- Multi-Tier Provider Fallback Mesh: If primary LLM (Gemini Flash) hits quota limits, traffic fails over seamlessly to Groq (Llama 3.1 70B), then OpenAI GPT-4o-mini, and finally a local heuristic parser.

GraphOne / FrontierAtlas - Technical Architecture & Scale Design

Production Design for 500,000+ Multi-Dimensional Intelligence Graph Ingestion

3. Freshness Tracking & Distributed Deduplication

Ensuring real-time news and job postings are strictly < 24-hours fresh and processed exactly once across distributed crawler nodes:

- Distributed Redis Bloom Filters: Before fetching full page payloads, crawler nodes check a centralized Redis Bloom Filter keyed by normalized canonical URL hash (URL-hash + PubDate). Bloom filters provide O(1) time complexity with 99.9% memory efficiency.
- 24-Hour Sliding TTL: Deduplication keys are assigned a strict 24-hour Sliding Time-To-Live (TTL), guaranteeing that content is checked afresh daily while preventing duplicate work across parallel crawler instances.
- Publication Timestamp Normalization: Extracted publication dates (including relative formats like '3 hours ago') are parsed into standard UTC ISO-8601 timestamps and filtered out if delta > 24 hours.

4. Multi-Modal Storage Architecture

GraphOne requires storing rich entity schemas alongside deep relational links and semantic vector embeddings:

- Primary RDBMS (PostgreSQL 16 + JSONB): Stores core entity metadata (Startups, Products, Papers, Jobs, News) with ACID compliance, schema versioning ('1.0'), and fast B-Tree indexing on canonical entity IDs.
- Vector Store (Qdrant / Pinecone): Stores dense semantic vector embeddings for research paper abstracts, product descriptions, and news content to enable RAG and similarity search.
- Graph Database (Neo4j / Memgraph): Maps complex graph relationships: (Startup)-[:BUILD_PRODUCT]->(Product), (Paper)-[:CORRELATED_WITH]->(GitHub_Repo), (Company)-[:POSTED_JOB]->(Job). Enables topological traversal for intelligence insights.

5. Dynamic Entity Canonicalization & Resolution

Messy raw entity strings (e.g. 'Open AI', 'OpenAI Inc.', 'OpenAI') are normalized using a hybrid approach:

1. Token Normalization: Legal suffix removal ('Inc', 'LLC', 'Corp') and string cleaning.
2. Exact Alias Matching: O(1) dictionary lookup against alias maps.
3. RapidFuzz Similarity: Token sort ratio matching against a 50-entity seed canonical database (threshold >= 82%).
4. Audit Logging: Every mapping event is recorded in entity_mapping_log.csv for transparent lineage tracking.